

Data Structures *Linked List*

Jim D. Pham

3/16/2016

What is data?



For example:

```
int main() {
```

```
...
```

```
int employeeId = 102;
```

```
...
```

```
}
```

■ Data

- A collection of facts/information that can be processed by a computer and uses for analysis.

■ Types of data

- bit, byte, number, text, audio, video, etc.

Employee ID: 102	
First Name:	John
Last Name:	Doe
Job:	CNET Instructor
Salary:	75000
Hire Date:	03/16/2016

...

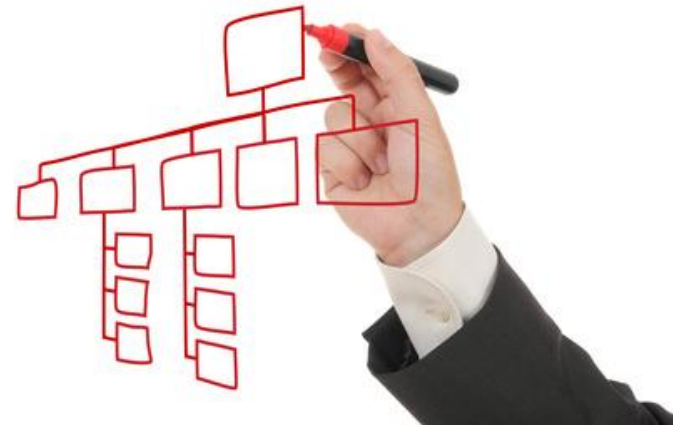
Division

Department

**These are examples
of Data Structures**

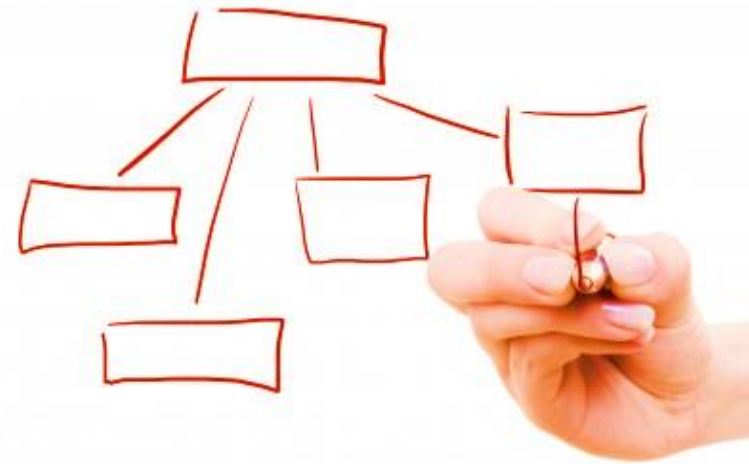
What is data structure?

- A particular way to storing and organizing data in a computer so that it can be used efficiently and effectively.
- Data structure is the logical model of a particular organization of data.
 - A combination of basic data types and/or structures.
 - e.g. boolean, char, integer, float, complex type
- A grouping of data elements grouped together.
 - e.g. an array of integers, array of structures



Why data structure?

- Associate certain problems with certain solutions
- Breaks down a complex problems into a simpler components
- Design and implement large-scale computer system to work efficiently



Good data structures and algorithms make program development easier and lead to well structured program.

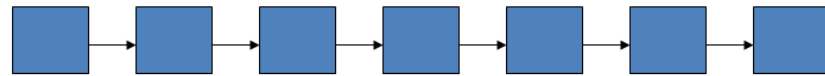
Common Types of Data Structures

- Array



Array

- Linked List



Linked List

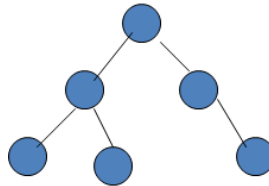
- Heap

- Hash Table

- Queue

- Stack

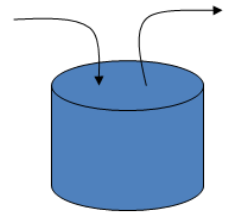
- Tree



Tree



Queue



Stack

Data Structures are containers.

Algorithms Linked List

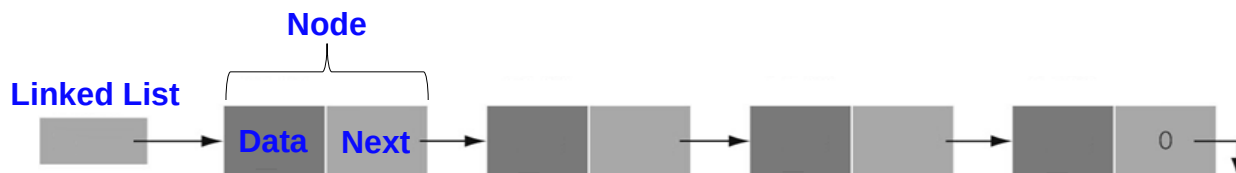
- Linked List is a sequence of links which contains items (nodes). Each link contains a connection to another link.

Node:

- **Data** – Each Link of a linked list can store a data called an element.
- **Next** – Each Link of a linked list contain a link to next link called Next.
- **LinkedList** – A LinkedList contains the connection link to the first Link called First.

Basic Operations:

- **Insertion**
- **Deletion**
- **Display**
- **Search**
- **Delete**



Basic Linked List Data Structure

Abstract Data Types

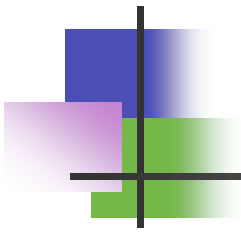
- An abstract data type (ADT) is a data type created by the programmer and is composed of one or more data types.

Structure of Linked List Nodes

```
9  // Definition of a linked list ADT
10 template <class E>
11 struct NodeType {
12     E data;
13     NodeType<E> *next;
14 };
```

Linked List





Linked List Implementation

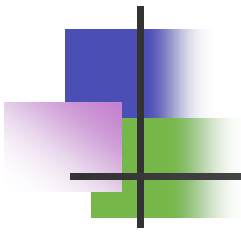
e.g. Employee ADT

```
// Let us consider the following
// structure definition of Employee
struct Employee {
    int empId;
    string firstName;
    string lastName;
    string job;
    string hireDate;
    int salary;
};
```

LinkedList Class

```
template <class E>
class LinkedList : public MyLinkedList<E>
{
public:
    bool search(const E& e) const;
    void addFirst(const E& e);
    void addLast(const E& e);
    void add(const E& e);
    void add(const int index, const E& e);
    void remove(const E& e);
    void removeAt(const int index);
    void displayList();
};
```

Declaration: `LinkedList<Employee> empList;`



Example of Employee LinkedList

- Declaration:

- `LinkedList<Employee> empList;`

- Basic Operations:

- `empList.add(emp)`

e.g. Employee **emp** = { 100, "John", "Doe", "CNET Instructor", 75000, "08/30/1999" } ;
empList.add(emp);

- `empList.add(index, emp)`

e.g. Employee **emp** = { 103, "Jim", "Pham", "CS Instructor", 150000, "08/23/2016 " } ;
empList.add(1, emp);

- `empList.remove(emp)`

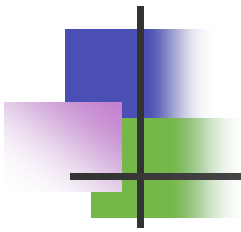
- `empList.removeAt(index)`

e.g. ***empList.removeAt(1);***

- `empList.displayList()`

- `LinkedListIterator iterator = empList.Iterator();`

i.e. *traverse the list, starting with the first node*

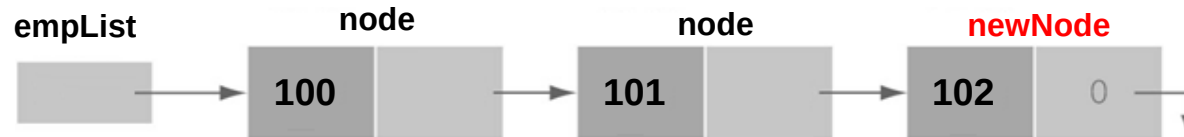


Algorithm for method **Add**

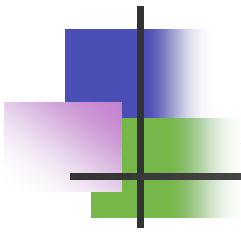
- 1) Create a new node
- 2) Store data in the new node
- 3) Insert the node into the list:
 - Case 1: Before the first node
 - Case 2: After the last node
 - Case 3: Add the node at the specified index

If you add a new node, then pointer needs to be adjusted and increment the list count by 1.

Add Node Implementation



```
122 template <class E>
123 void LinkedList<E>::add(const E& e) {
124     NodeType<E> *newNode;           //pointer to create the new node
125     newNode = new NodeType<E>;      //create the new node
126     newNode->data = e;               //store the new item in the node
127     newNode->next = nullptr;         //set the link field of newNode to nullptr
128
129     //if the list is empty, newNode is both the first and last node
130     if (first == nullptr) {
131         first = newNode;
132         last = newNode;
133     }
134     else {
135         //the list is not empty, insert newNode after last
136         last->next = newNode;        //insert newNode after last
137         last = newNode;              //make last point to the actual last node in the list
138     }
139     count++;                         //increment count
140 }
```



Algorithm for method **Remove**

- 1) The list is empty.
- 2) The node to be deleted is:
 - a. the first node.
 - b. some where in the list.
 - c. not the last node.
 - d. the last node.
 - e. not in the list.

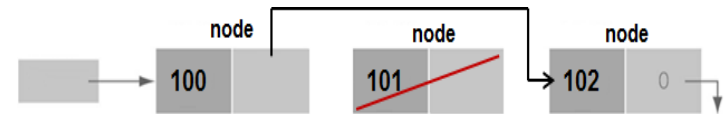
If you remove a new node, then pointer needs to be adjusted and decrement the list count by 1.

Remove Node Implementation

```

164 template <class E>
165 void EmployeeLinkedList<E>::remove(const E& e) {
166     NodeType<E> *current;           //pointer to traverse the list
167     NodeType<E> *trailCurrent;       //pointer just before current
168     if (first == nullptr) {
169         cout << "Cannot delete from an empty list." << endl;
170     } else if (((Employee)first->data).empId == ((Employee) e).empId) {
171         current = first;
172         first = first->next;
173         count--;
174         if (first == nullptr)        //the list has only one node
175             last = nullptr;
176         delete current;
177     } else { //search the list for the node with the given info
178         bool found = false;
179         trailCurrent = first;         //set trailCurrent to point to the first node
180         current = first->next;        //set current to point to the second node
181         while (current != nullptr) {
182             if ( ((Employee)current->data).empId == ((Employee)e).empId ) {
183                 found = true;
184                 break;
185             }
186             else {
187                 trailCurrent = current;
188                 current = current->next;
189             }
190         }
191         if (found) { // delete the node
192             trailCurrent->next = current->next;
193             if (last == current)      //node to be deleted was the last node
194                 last = trailCurrent; //update the value of last
195             delete current;           //delete the node from the list
196             count--;
197         }
198         else
199             cout << "The item to be deleted is not in the list." << endl;
200     }
201 } //end remove

```



Test Driver Program for Employee Linked List

```
15 int testLinkedList() {
16     LinkedList<Employee> empList;
17     Employee emps[] = {
18         { 100, "John", "Doe", "CNET Instructor", 75000.00, "08/30/1999" },
19         { 101, "Adam", "Smith", "Psychology Instructor", 120000.00, "07/31/2005" } };
20     cout << "Add new record to linked list" << endl;
21     for each (Employee e in emps){
22         empList.add(e);
23     }
24     cout << "Add new record at index 1:" << endl;
25     Employee e1 = { 102, "Jim", "Pham", "CS Instructor", 150000.00, "08/23/2016" };
26     empList.add(1, e1);
27     cout << "Record to be removed:" << endl;
28     Employee e2 = empList.get(2);
29     empList.remove(e2);
30     cout << "Display linked list after remove..." << endl;
31     empList.displayList();
32     return 0;
33 }
```

0	100	John	Doe	CS Instructor	08/30/1999	75000
1	102	Jim	Pham	CNET Instructor	08/23/2016	150000
2	101	Adam	Smith	Psychology Instructor	07/31/2005	120000

Line 22: Add 2 employees at index 0 and 1.

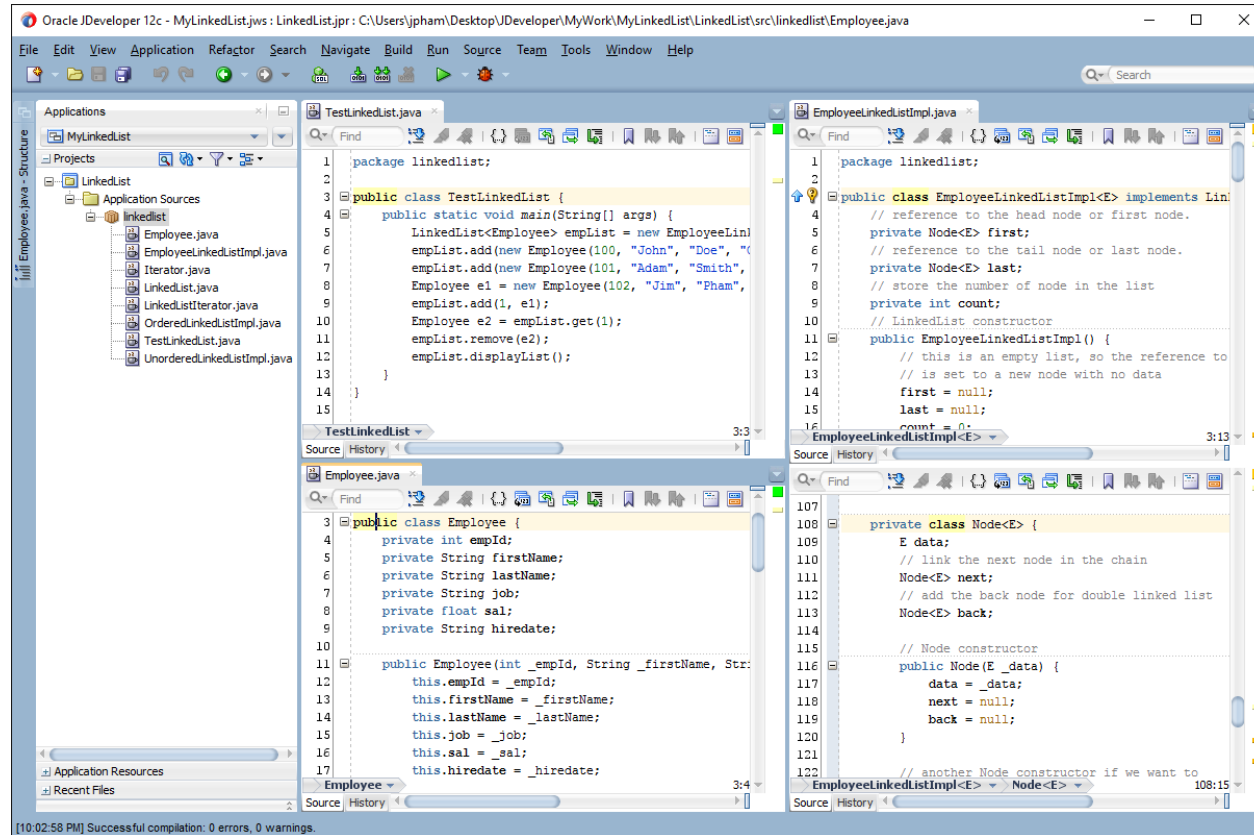
Line 26: Add another employee at index 1

➤ **Now employee 101 is at index 2**

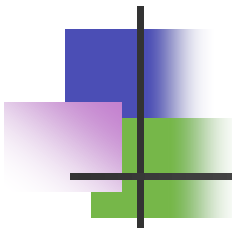
➤ **And employee 102 is at index 1**

Line 29: Remove employee at index 2

Linked List Implementation in Java



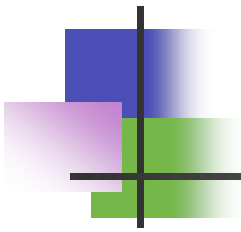
Linked list can be implemented using any OOP.



ADTs and Data Structures

in Programming Languages

- Modern programming languages usually have a library of data structures
- C++ standard template library
- Java collections framework
- .NET framework
- Other OOP languages have similar collections framework



Summary

- Basic data structures
- The important of data structure
- Data Structure and Algorithms
- Abstract Data Type (ADT)
- Basic ***LinkedList*** implementation
- Basic operations ***Add*** and ***Remove***

Questions

